Deep Reinforcement Learning

Introduction

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Who am I?

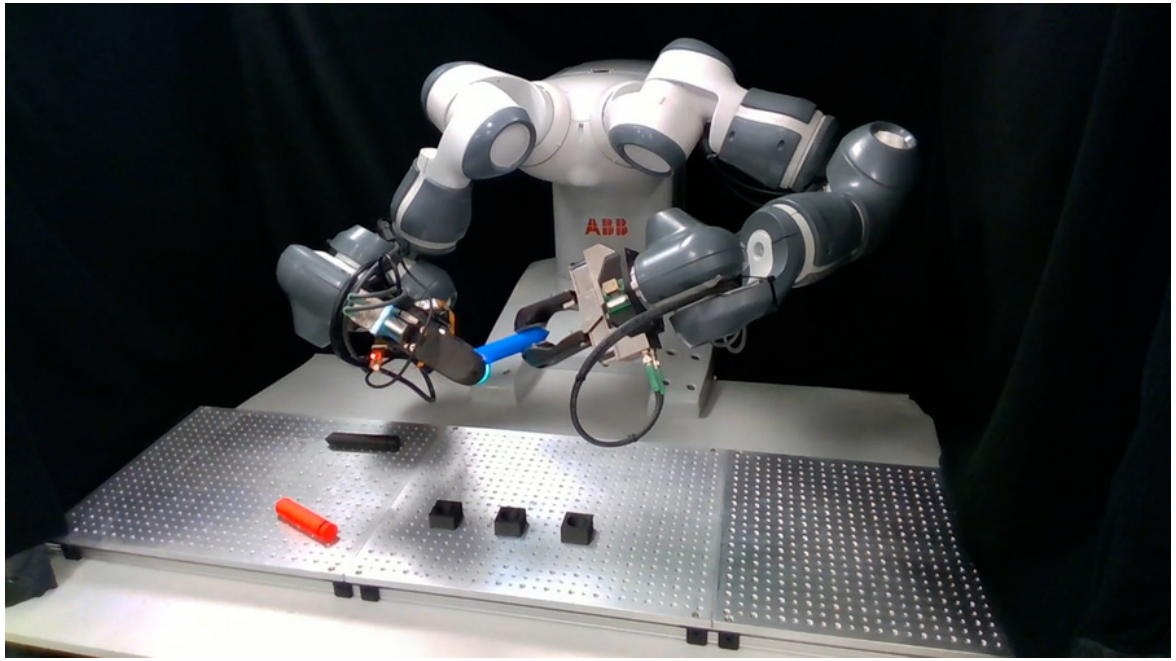
- Prof. Dr. Sebastian Peitz
- Chair of Safe Autonomous Systems
- JvF25, Room 124
- https://sas.cs.tu-dortmund.de/

The team

- Lecture: Sebastian Peitz
- Exercise: Konstantin Sonntag
- Additional team members working on the exercises
  - Jannis Becktepe
  - Septimus Boshoff
  - Hans Harder
  - Christian Mugisho Zagabe
  - Stefan Weigand

Contents of the Lecture

Sequential decision making: Taking actions that affect our environment



Task: pick up pens and place them upright [Source]

Option 1: Rule-based system

- manual design of actions, given some sensor input (such as a camera)

Option 2: Model-based control

- given some model of the dynamics, optimize for the best possible action to take

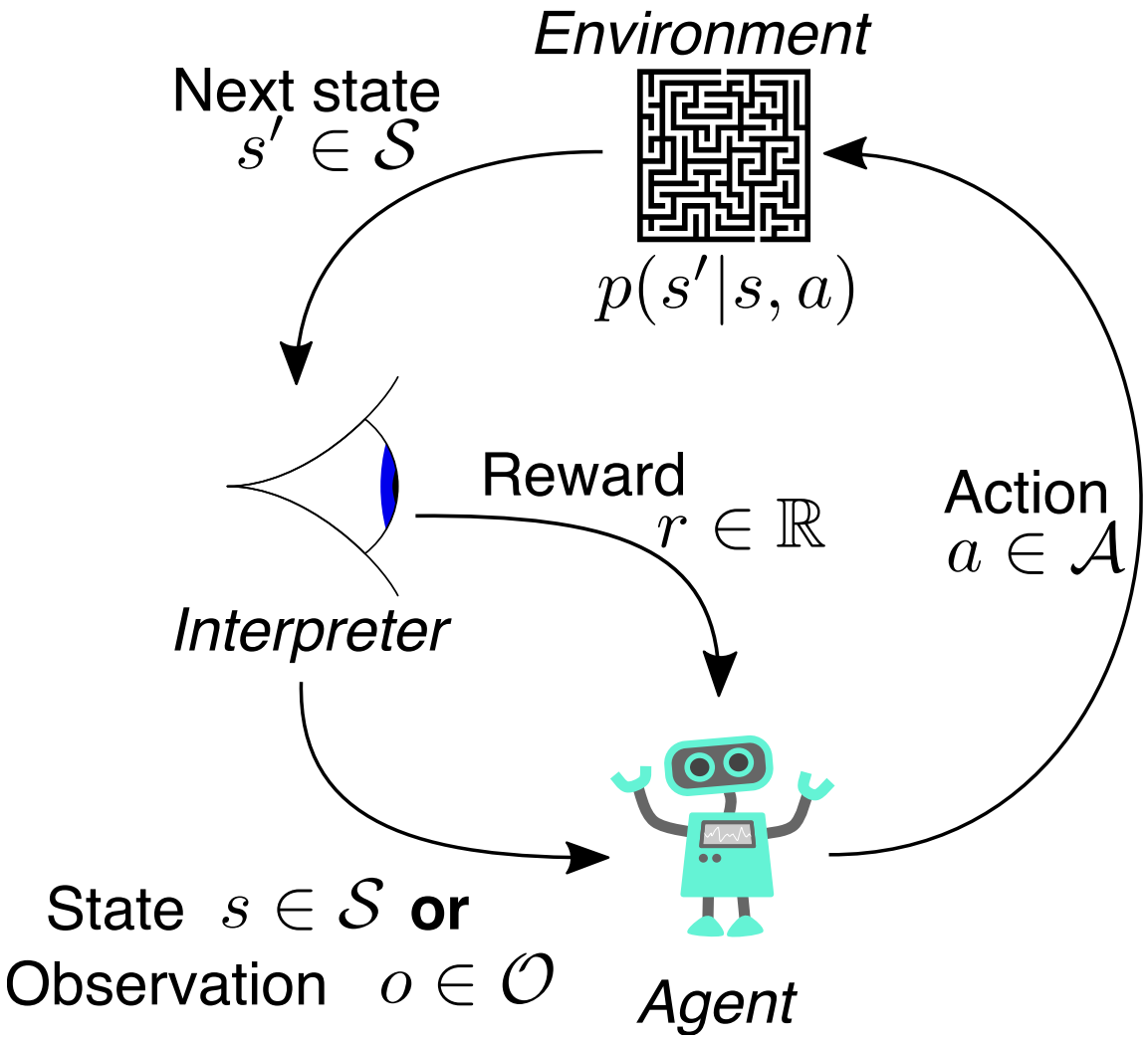
Option 3: Imitation learning

- collect data from experts and try to *clone the behavior*

Option 4: Reinforcement learning

- collect data in a *trial-and-error* fashion, improve via feedback

What is reinforcement learning?

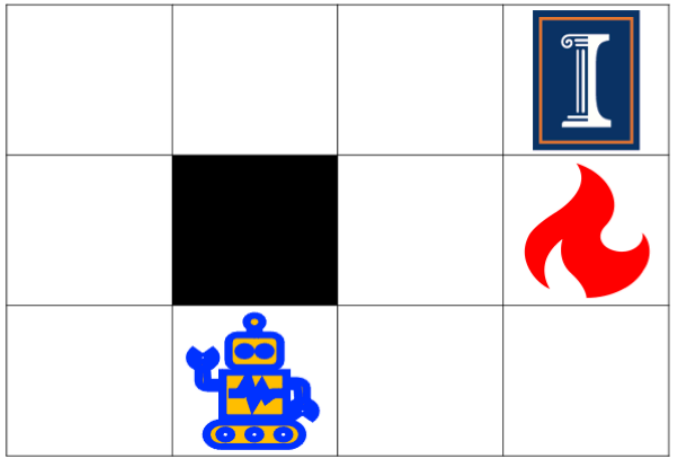


In words

- An **agent** perceives the state s of its environment.
- It takes an **action** a according to a **policy** $\pi(a|s)$.
- The environment changes due to the action: $s \to s'$.
- The agent receives a **reward** r.

The goal

Reinforcement learning examples (1)



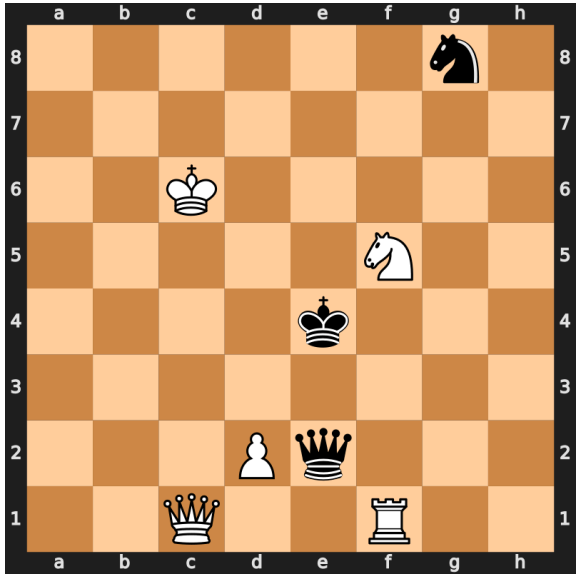
Grid world

- **Environment**: The grid world
- **Agent**: The robot
- **State** s: The robot's position
- **Action** a: Move left / right / up / down
- **Dynamics** $p(s'|s,a)$: The robot's new position...
  - ...deterministically, if the robot does exactly what the action demands
  - ...stochastically, if there is some noise in the taken action
- **Reward**:

$$r = \begin{cases} +1 & \text{if you reach the target field (top right)} \\ -1 & \text{if you hit the fire} \\ -0.1 & \text{otherwise} \end{cases}$$

- **Policy** π: The strategy according to which you move

Reinforcement learning examples (2)

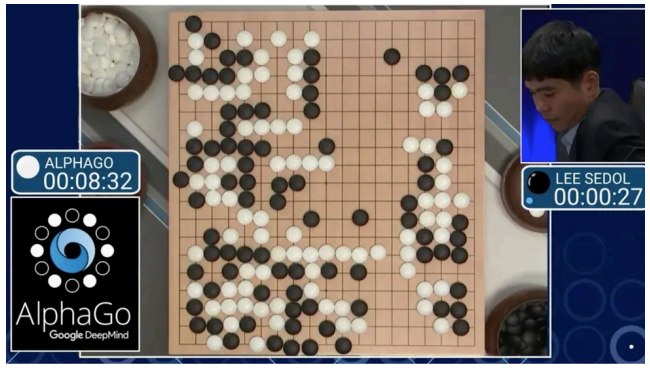


Chess board [Source]

- **Environment**: The chess board
- **Agent**: The chess player(s)
- **State** s: The board position
- **Action** a: Move one of your figures
- **Dynamics** $p(s'|s,a)$: The new board position…
  - …after your own and your opponent's moves $\longrightarrow$ your turn again
  - …after your move; it's the other players turn $\longrightarrow$ two-agent game
- **Reward**:

$$r = \begin{cases} +1 & \text{if you take your opponent's king} \\ -1 & \text{if your king is taken} \\ 0 & \text{otherwise} \end{cases}$$

- **Policy** π: The strategy with which you play



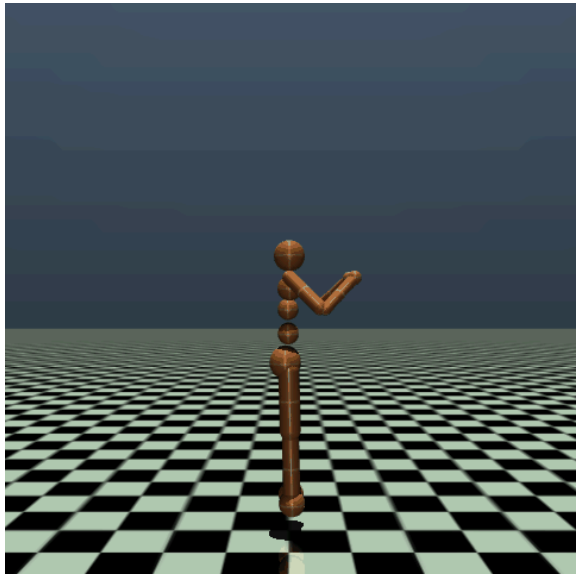
AlphaGo (Silver et al. 2016) [Source]

Reinforcement learning examples (3)

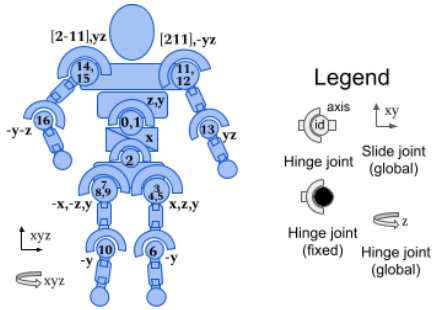
Pendulum [Source]

- **Environment**: A pendulum under gravitational force
- **Agent**: The control system trying to perform a swing-up
- **State** s: $\sin$ and $\cos$ of the angle φ (top: $\varphi = 0$) and angular velocity $\omega = \frac{d\varphi}{dt}$
- **Action** a: the applied torque
- **Dynamics** $p(s'|s, a)$: a differential equation describing the dynamics (*Newtonian mechanics*) $\rightarrow$ deterministic system
- **Reward**: $r = -(\varphi^2 + 0.1 \cdot \omega^2 + 0.001 \cdot a^2)$
- **Policy** π: the torque realizing the swing-up

Reinforcement learning examples (4)



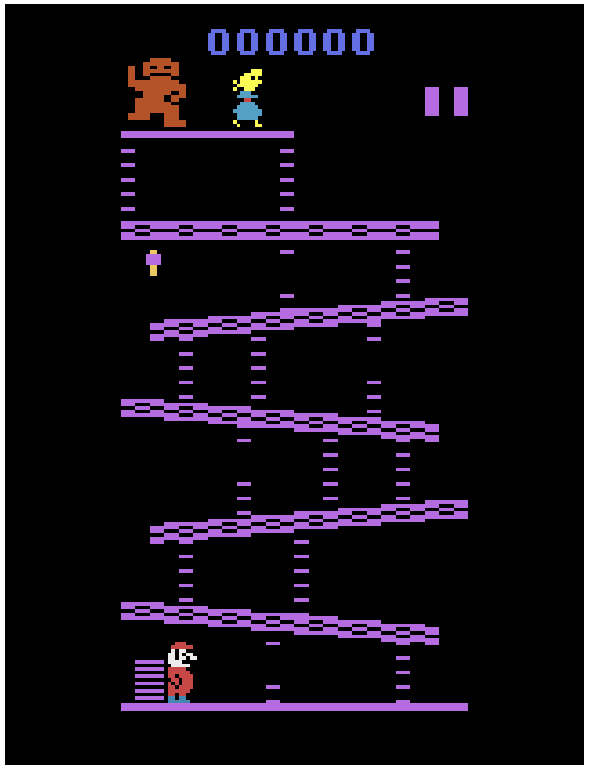
Humanoid [Source]



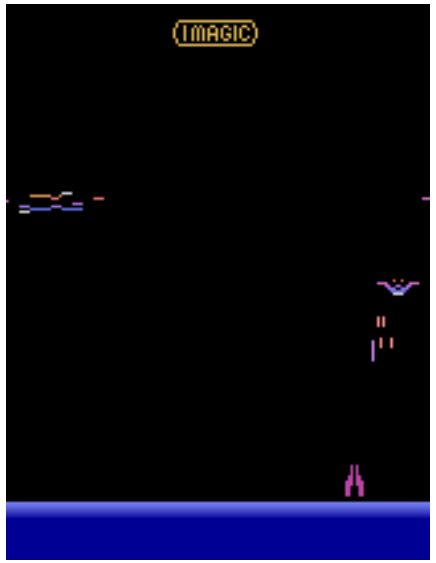
Model

- **Environment**: A robot moving on a planar ground
- **Agent**: The control system for actuating the robot
- **State** s: Positions and velocities of the components (45 in total)
- **Action** a: The torques applied to the 17 joints
- **Dynamics** $p(s'|s,a)$: a differential equation describing the dynamics (*Newtonian mechanics*) $\longrightarrow$ deterministic system
- **Reward**: healthy reward + forward reward - control cost - contact cost
- **Policy** π: Run as far as you can!

Reinforcement learning examples (5)



Atari: Donkey Kong [Source]

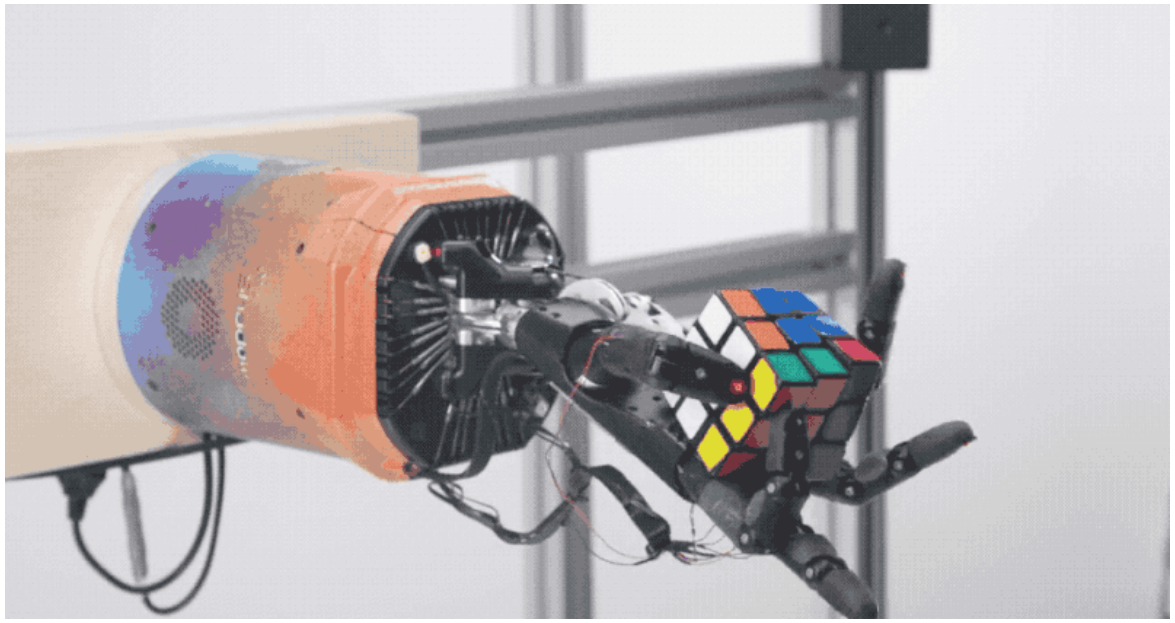


Atari: Demon Attack [Source]

- **Environment**: Atari video games
- **Agent**: The "player"
- **State** s: the pixels of the screen
- **Action** a: the game pad (i.e., moving, firing, etc.)
- **Dynamics** $p(s'|s,a)$: the video game reacting to the player's actions
- **Reward**: achieving the game's goals (stay alive, reach a target, eliminate opponents, ...)
- **Policy** π: play such that you achieve your goal as best as possible

Reinforcement learning in robotics

Boston Dynamics "Atlas" [Source]



OpenAI Rubik's Cube [Source]

The two central tasks in RL

In all chapters, we will distinguish between **two core learning tasks in reinforcement learning**:

1. **The prediction problem** (*evaluation*)

- For a fixed policy π, we want to **estimate the quality of a state** s (i.e., its **value**).

2. **The control problem** (*improvement*)

- We want to **improve our policy** π towards the optimal π^* that maximizes the value.
- This usually requires a back-and-forth between 1. and 2.
  - Given our estimate of the value, we improve the policy: $\pi' > \pi$.
  - We then need to reiterate the values given the new policy π' and so on.

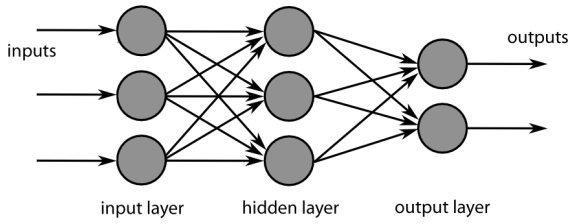
The key difference to supervised learning

- **Supervised learning**:
  - minimize error on a training dataset $\mathcal{D} = \{(x_1, y_1), \dots, (x_K, y_k)\}$:

$$\min_{\theta} \sum_{k=1}^{K} \|y_k - f_\theta(x_k)\|_2^2$$

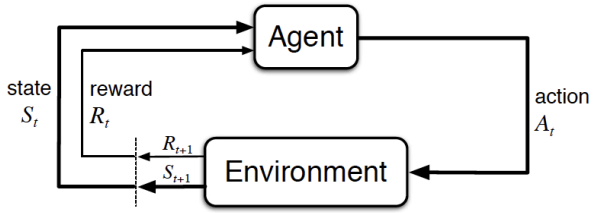
  - data is generally drawn independently and identically distributed (i.i.d.):

$$(x_k, y_k) \sim p(\mathcal{D}).$$



Feed forward neural network

- **Reinforcement learning**:
  - no supervisor, just a *reward signal*
  - feedback is delayed, not instantaneous
  - time really matters (sequential, non i.i.d. data)



Reinforcement learning (Sutton and Barto 1998)

- agents actions affect the subsequent data it receives

The big questions

- How can we learn a good policy π *from experience*?
- How can we address the *exploration-exploitation dilemma*?
- How do we use *deep learning* to address otherwise intractable problems?
- Which techniques can we use to facilitate *efficient and stable learning*?
- What are interesting *applications of RL*?
- What are the *open questions in RL research?*

Goal of the lecture

By the end of the course, we will

- understand the **theoretical basis** behind RL
- know the **landscape of RL algorithms** as well as their pros and cons
- be able to **implement RL algorithms** (using python)
- be capable to **critically assess** scientific papers as well as practical RL methods
- know how to tackle **real-world problems** using RL

Chapters

- **Introduction**
- **Multi-armed bandits**
- **Basics & tabular methods**
  - Markov Decision Processes (MDPs)
  - Dynamic Programming
  - Monte Carlo Methods
  - Temporal Difference Learning & Q-learning
- **Deep-learning-based methods**
  - Brief introduction to deep learning
  - Deep Q-learning
  - Actor-Critic algorithms
- **Model-based control**
  - Planning & optimal control
  - Model-based RL
  - Exploration
- **Advanced topics**
  - Offline RL
  - Transfer learning
  - Multi-agent RL
  - Behavior cloning

Organization

Lectures & Exercises

Lectures

- Tuesdays (10:15 - 11:45, OH16/205)
- Wednesdays (08:30 - 10:00, OH12/4.031)

Exercises

- Thursdays (10:15 - 11:45, OH16/205)
- One exercise sheet per week
  - pen-and-paper exercises the cover and extend the theoretical content of the lecture
  - programming exercises (in Python) supporting the Studienleistung (next slide)

Studienleistung – Programming tasks

- over the semester, we will publish four long-term programming tasks
  - they cover 3-4 worth of content weeks
  - the programming exercises on the weekly exercise sheets cover parts of these programming tasks
  - to be accepted to the exam, **you need to pass at least 3/4**
- task evaluation
  - the tasks are pass/fail
  - we will take them into consideration in the final exam by dedicating several questions to your submissions
  - this way, they will account for $\approx \frac{1}{3}$ of your final grade
- usage of ChatGPT, Gemini, Claude, etc.
  - is allowed 😊

Exam

Format:

- oral exam during the semester break (dates to be announced soon)
- 30 minutes per candidate

Content:

- all topics covered in the lectures
- all topics covered in the exercises
- in-depth questions regarding your submitted programming tasks regarding
  - your choices regarding modeling / implementation
  - your results
  - your usage (e.g. prompts) of LLMs, if applicable

Literature

Reinforcement learning

- Reinforcement learning: an introduction (Sutton and Barto 1998)
- David Silver's RL lecture (Silver 2015)

Mathematical basics

- Linear Algebra: Stanford CS229 review material
- Probability:
  - Stanford CS229 review material
  - Introduction to Probability lecture notes by Dimitri Bertsekas

Programming

- Berkeley's CS285 pyTorch course (Google Colab)

References

Silver, David. 2015. “Lectures on Reinforcement Learning.” url: https://www.davidsilver.uk/teaching/.

Silver, David, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, et al. 2016. “Mastering the Game of Go with Deep Neural Networks and Tree Search.” *Nature* 529 (7587): 484–89.

Sutton, R. S., and A. G. Barto. 1998. *Reinforcement Learning: An Introduction*. MIT Press.